Name: Nhem Sun Serinyuth

Gen11 | Cyber | G2

ID: IDTB110255

OS – Lab – Bash Script 3 and Deadlock

Level1:

```
g11-nhem-serinyuth@cbsecg11-cadtos:~/os-lab-deadlock$ df -h | grep loop
/dev/loop96      5.4M   24K  4.7M   1% /media/g11-deng-monyoudom/889a55e2-1e04-4966-948e-e6d207258754
/dev/loop104     5.4M   24K  4.7M   1% /media/g11-deng-monyoudom/ae46ad1d-9f6b-401c-bb39-99abb6fbdb59
/dev/loop0       5.4M   24K  4.7M   1% /media/g11-ry-sopanha/e8e8c827-9d8e-4e4a-8fd0-f3f28bfcaf00
/dev/loop21      5.4M   24K  4.7M   1% /media/g11-ry-sopanha/b2aeb2e2-8e21-4e29-b4cd-720f58cae8fc
/dev/loop6       5.4M   24K  4.7M   1% /media/g11-loun-rotha/7a73f39b-f496-469f-ae78-218d528541ec
/dev/loop22      5.4M   24K  4.7M   1% /media/g11-loun-rotha/ef7474f8-db25-4515-8e79-49f1f6a12447
/dev/loop29      5.4M   24K  4.7M   1% /media/g11-ny-chamnan/25ac78be-514e-4190-b097-9301281542ef
/dev/loop30      5.4M   24K  4.7M   1% /media/g11-ny-chamnan/94fd0c73-ff7e-4c66-b33c-d8445df2a123
/dev/loop31      5.4M   24K  4.7M   1% /media/g11-kong-monineath/d376ccb9-6204-4963-a423-4c65b56f70fd
/dev/loop32      5.4M   24K  4.7M   1% /media/g11-kong-monineath/f4a3b36e-810a-4253-af79-2499fe77c0ed
/dev/loop38      5.4M   24K  4.7M   1% /media/g11-ny-chamnan/25ac78be-514e-4190-b097-9301281542ef1
/dev/loop39      5.4M   24K  4.7M   1% /media/g11-ny-chamnan/94fd0c73-ff7e-4c66-b33c-d8445df2a1231
/dev/loop33      5.4M   24K  4.7M   1% /media/g11-oeun-vanmonireak/147c0abf-3ea4-4356-a044-237dddab3da4
/dev/loop40      5.4M   24K  4.7M   1% /media/g11-oeun-vanmonireak/a66576c9-d73f-457a-9400-a5c8ce3aaa13
/dev/loop44      5.4M   24K  4.7M   1% /media/g11-nhem-serinyuth/ec7a0294-6105-4781-b7f4-e8dbe3432e17
/dev/loop45      5.4M   24K  4.7M   1% /media/g11-nhem-serinyuth/4c1c61e3-ad83-426b-b328-3089f30427cc
g11-nhem-serinyuth@cbsecg11-cadtos:~/os-lab-deadlock$
```

Explanation: The df -h | grep loop output shows two loopback devices (/dev/loop44 and /dev/loop45) successfully mounted and active on the system. This proves that the two virtual drive image files (vault_alpha.img and vault_beta.img) have been correctly attached to loop devices and formatted with a working ext4 filesystem, because the OS is now able to report their size, used space, and available space — just as it would for a real physical hard drive.

Level3:

```
g11-nhem-serinyuth@cbsecg11-cadtos:~/bin$ sync_up
[Sync_UP] Locking Vault Alpha...
[Sync_UP] Vault Alpha locked. Computing data delta...

[Sync_UP] Locking Vault Beta to write...
```

```
g11-nhem-serinyuth@cbsecg11-cadtos:~/bin$ sync_down
[Sync_DOWN] Locking Vault Beta...
[Sync_DOWN] Vault Beta locked. Computing data delta...
[Sync_DOWN] Locking Vault Alpha to write...
```

```
g11-nhem-serinyuth@cbsecg11-cadtos:~$ ps aux | grep sync_
root           5  0.0  0.0      0     0 ?        I<   Feb02   0:00 [kworker/R-sync_wq]
g11-nhe+ 1041899  0.0  0.0   9940  3688 pts/26   S+   13:43   0:00 /bin/bash /home/g11-nhem-serinyuth/bin/sync_up
g11-nhe+ 1041900  0.0  0.0   9940  2316 pts/26   S+   13:43   0:00 /bin/bash /home/g11-nhem-serinyuth/bin/sync_up
g11-nhe+ 1041906  0.0  0.0   9940  3700 pts/18   S+   13:43   0:00 /bin/bash /home/g11-nhem-serinyuth/bin/sync_down
g11-nhe+ 1041907  0.0  0.0   9940  2272 pts/18   S+   13:43   0:00 /bin/bash /home/g11-nhem-serinyuth/bin/sync_down
```

Explanation: Both sync_up and sync_down froze indefinitely and never reached "Sync complete." sync_up held the Alpha lock (fd 200) and was waiting for the Beta lock (fd 201). sync_down held the Beta lock (fd 201) and was waiting for the Alpha lock (fd 200). Since neither process releases its held lock until it acquires the second one, and neither can acquire the second one because the other process holds it, both processes are stuck forever in a circular wait — the classic deadlock condition.

Level4:

```
g11-nhem-serinyuth@cbsecg11-cadtos:~/bin$ cross_sync_alpha & cross_sync_beta
[1] 1048159
[CrossSync-B] Locking my vault (Beta)...
[CrossSync-A] Locking my vault (Alpha)...
[CrossSync-A] My vault locked. Sleeping 2s...
[CrossSync-B] My vault locked. Sleeping 2s...
[CrossSync-B] Reaching for Alpha vault...
[CrossSync-A] Reaching for Beta vault...
```

Explanation: I simulated the distributed deadlock alone by creating two directories — public_dr_alpha and public_dr_beta — each with their own vault.lock file, acting as two separate sites.

When cross_sync_alpha and cross_sync_beta ran simultaneously, cross_sync_alpha held Alpha's lock and waited for Beta's, while cross_sync_beta held Beta's lock and waited for Alpha's. Both scripts froze forever — neither could proceed because each was waiting for a lock the other was holding.

This simulates a distributed denial of service where two systems freeze each other entirely due to a locking design flaw, not an external attack.

Level5:

```
g11-nhem-serinyuth@cb
g11-nhem-serinyuth@cbsecg11-cadtos:~/bin$ cross_sync_alpha & cross_sync_beta
[1] 1057729
[CrossSync-A] Locking my vault (Alpha)...
[CrossSync-B PATCHED] Acquiring Alpha lock first (global order)...
[CrossSync-A] My vault locked. Sleeping 2s...
[CrossSync-A] Reaching for Beta vault...
[CrossSync-A] Both vaults locked. Syncing...
[CrossSync-A] Done.
[CrossSync-B PATCHED] Alpha locked. Now locking Beta...
[CrossSync-B PATCHED] Both locked. Syncing...
[CrossSync-B PATCHED] Done. Releasing locks.
[1]+  Done                    cross_sync_alpha
g11-nhem-serinyuth@cbsecg11-cadtos:~/bin$
```

Explanation: After patching cross_sync_beta to always acquire Alpha's lock first, both scripts ran simultaneously without deadlocking. One script acquired Alpha's lock and ran to completion while the other waited. Once the first finished and released its locks, the second acquired them and completed successfully.

The circular wait was broken because both scripts now request locks in the same global order — Alpha always before Beta. It is impossible for a deadlock cycle to form when all processes compete for the same first resource.

Level6:

```
g11-nhem-serinyuth@cb
g11-nhem-serinyuth@cbsecg11-cadtos:~/bin$ sync_up
[Sync_UP] Locking Vault Alpha...
[Sync_UP] Vault Alpha locked. Computing data delta...
[Sync_UP] Locking Vault Beta to write...
[Sync_UP] Vault Beta locked. Syncing data...
[Sync_UP] Sync complete. Releasing locks.
g11-nhem-serinyuth@cbsecg11-cadtos:~/bin$
```

```
g11-nhem-serinyuth@cb
g11-nhem-serinyuth@cbsecg11-cadtos:~/bin$ sync_timeout
[Sync_Timeout] Attempting to lock Vault Alpha (Timeout: 5s)...
[ERROR] Lock timeout! Aborting sync to prevent deadlock.
g11-nhem-serinyuth@cbsecg11-cadtos:~/bin$
```

Explanation: While sync_up was holding Alpha's lock, sync_timeout tried to acquire it but gave up after 5 seconds and printed an error message instead of freezing.

This is useful because a process that automatically aborts frees up system resources immediately, keeping the server responsive without needing manual intervention.

Level7:

```
g11-nhem-serinyuth@cb
g11-nhem-serinyuth@cbsecg11-cadtos:~/bin$ teardown
Unmounting virtual vaults...
Unmounted /dev/loop44.
Unmounted /dev/loop45.
Teardown complete.
```

```
g11-nhem-serinyuth@cbsecg11-cadtos:~/bin$ df -h | grep loop
/dev/loop96     5.4M   24K  4.7M   1% /media/g11-deng-monyoudom/889a55e2-1e04-4966-948e-e6d207258754
/dev/loop104    5.4M   24K  4.7M   1% /media/g11-deng-monyoudom/ae46ad1d-9f6b-401c-bb39-99abb6fbdb59
/dev/loop21     5.4M   24K  4.7M   1% /media/g11-ry-sopanha/b2aeb2e2-8e21-4e29-b4cd-720f58cae8fc
/dev/loop6      5.4M   24K  4.7M   1% /media/g11-loun-rotha/7a73f39b-f496-469f-ae78-218d528541ec
/dev/loop22     5.4M   24K  4.7M   1% /media/g11-loun-rotha/ef7474f8-db25-4515-8e79-49f1f6a12447
/dev/loop29     5.4M   24K  4.7M   1% /media/g11-ny-chamnan/25ac78be-514e-4190-b097-9301281542ef
/dev/loop30     5.4M   24K  4.7M   1% /media/g11-ny-chamnan/94fd0c73-ff7e-4c66-b33c-d8445df2a123
/dev/loop31     5.4M   24K  4.7M   1% /media/g11-kong-monineath/d376ccb9-6204-4963-a423-4c65b56f70fd
/dev/loop32     5.4M   24K  4.7M   1% /media/g11-kong-monineath/f4a3b36e-810a-4253-af79-2499fe77c0ed
/dev/loop38     5.4M   24K  4.7M   1% /media/g11-ny-chamnan/25ac78be-514e-4190-b097-9301281542ef1
/dev/loop39     5.4M   24K  4.7M   1% /media/g11-ny-chamnan/94fd0c73-ff7e-4c66-b33c-d8445df2a1231
/dev/loop48     5.4M   24K  4.7M   1% /media/g11-thon-sopheaktra/3b2ab07b-ee2f-4eab-aa36-f27667182c31
/dev/loop49     5.4M   24K  4.7M   1% /media/g11-thon-sopheaktra/2b0180ac-4592-4064-a40c-48c4409f9c0a
/dev/loop47      56M   24K   52M   1% /media/g11-hean-chhanun/VAULT_ALPHA
/dev/loop55     5.4M   24K  4.7M   1% /media/g11-sin-hong/3773d51f-2666-49d1-b8e0-ad478e96dbc0
```

*Notes: (No more loop44, loop45)

Explanation: After running teardown, df -h | grep loop no longer shows any mounted loop devices for my account. Both virtual drives were cleanly unmounted and their loop devices detached from the kernel.

Proper teardown is critical because unmounting while data is being written can corrupt the filesystem. Leaving orphaned loop devices also wastes kernel resources that persist until the server reboots.

Git-Repo: https://github.com/SERINYUTH/os-lab-deadlock-IDTB110255.git